



Procesamiento Digital de Señales

Ingeniería en Computación

Laboratorio 1

Muestreo con MCXN947

FRDM-MCXN947 · MCUXpresso IDE · SDK MCUXpresso

Integrante	Matrícula	Correo
Costamagna, Matías	44.512.437	matias.costamagna@mi.unc.edu.ar
Moreyra, Julián	45.214.537	julian.moreyra@mi.unc.edu.ar
Rivera, Mariano	[matrícula]	luismarianorivera.25@mi.unc.edu.ar
Rojas, Facundo	[matrícula]	[correo]

Docentes: Rossi, Roberto Raúl · Parlanti, Gustavo Fernando · Molina, Germán Alcides

Fecha de entrega: 27 de septiembre de 2026

Índice

1	Introducción	2
1.1	Requerimientos	2
1.2	Correspondencia con la presentación	2
2	Configuración del ambiente	3
2.1	Herramientas utilizadas	3
2.2	Creación del proyecto	3
2.3	Relojes	3
2.4	Pines	5
2.5	Periféricos	5
3	Diseño de la aplicación	11
3.1	Arquitectura general	11
3.2	Generación de las velocidades de muestreo	11
3.3	Máquina de estados	12
3.4	Formato q15 y conversiones	12
3.5	Rutinas de interrupción	13
4	Implementación	14
4.1	Constantes y variables globales	14
4.2	Programa principal	14
4.3	Cambio de velocidad	15
4.4	Rutinas de interrupción	16
5	Verificación	18
5.1	Procedimiento	18
5.2	Frecuencia de muestreo medida	18
5.3	Señal de entrada y salida	18
5.4	Cumplimiento de los requerimientos	19
6	Conclusiones	20
6.1	Mejoras posibles	20
A	Código fuente	21

1 Introducción

Laboratorio 1 — Muestreo con MCXN947

Objetivo. Manejar el módulo del conversor A/D integrado en el MCU MCXN947.

Descripción. Realizar un programa aplicativo que sea capaz de digitalizar una señal a través del módulo del ADC disponible en la placa MCXN947 a distintas velocidades de muestreo; las velocidades requeridas son 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s. Los cambios de las velocidades de muestreo serán realizados con una de las teclas de la placa de evaluación, en forma de un buffer circular. Cada velocidad de muestreo se indicará a través de un color RGB del LED. Con otra tecla de la placa se habilitará la adquisición o se parará la misma (Run/Stop). Los valores adquiridos serán almacenados en memoria en un buffer circular de 512 muestras del tipo q15 (fraccional de 15 bits) y a su vez serán enviados a través del DAC (de 12 bits).

Presentación.

1. Mostrar el programa corriendo.
2. Subir un archivo conteniendo lo siguiente:
 - Configuración del ambiente (pantallas con timers, ADC, etc.).
 - Diagrama de diseño de la aplicación explicando sus partes.
 - Código fuente de la misma con todo el proyecto.

1.1 Requerimientos

De la consigna se desprenden los siguientes requerimientos, que se usan como guía para el diseño y como lista de verificación en la sección 5:

- R1** Muestrear una señal analógica con el ADC a 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s.
- R2** Recorrer las velocidades en forma circular con una tecla de la placa.
- R3** Indicar la velocidad activa con un color del LED RGB.
- R4** Habilitar o detener la adquisición con otra tecla (Run/Stop).
- R5** Guardar las muestras en un buffer circular de 512 muestras en formato q15.
- R6** Enviar cada muestra al DAC de 12 bits.

1.2 Correspondencia con la presentación

Tabla 1: Dónde se encuentra cada ítem pedido en la presentación.

Ítem pedido	Ubicación en este informe
Configuración del ambiente	Sección 2
Diagrama de diseño de la aplicación	Sección 3
Código fuente del proyecto	Sección 4 y Anexo A
Programa corriendo	Demostración en clase; evidencia en sección 5

2 Configuración del ambiente

2.1 Herramientas utilizadas

Tabla 2: Hardware y software del entorno de trabajo.

Elemento	Versión / modelo
Placa de evaluación	FRDM-MCXN947 (Cortex-M33, núcleo <code>cm33_core0</code>)
IDE	MCUXpresso IDE v25.6.136
SDK	SDK_26.06.00_FRDM-MCXN947
Herramientas de config.	MCUXpresso Config Tools (Clocks, Pins, Peripherals)
Proyecto base	Ejemplo <code>lpadc_interrupt_cm33_core0</code> del SDK
Bibliotecas	Drivers del SDK y CMSIS-DSP (tipo <code>q15_t</code>)
Generador de señales	[marca y modelo]
Osciloscopio	[marca y modelo]

2.2 Creación del proyecto

Para crear el proyecto TP1 se importó del SDK de la FRDM-MCXN947 el ejemplo `lpadc_interrupt`, que ya trae la inicialización de la placa y el driver del LPADC. Sobre esa base:

1. Se instaló el SDK de la FRDM-MCXN947 en el IDE y se importó el ejemplo para el núcleo `cm33_core0`.
2. Se agregaron los drivers `ctimer`, `dac` (LPDAC), `inputmux`, `gpio`, `vref` y `spc`, y la biblioteca CMSIS-DSP para disponer del tipo `q15_t` (`arm_math.h`).
3. Con Config Tools se configuraron los relojes, los pines y los periféricos; el código generado quedó en `board/clock_config.c`, `board/pin_mux.c` y `board/peripherals.c`.
4. La lógica de la aplicación se escribió en `source/TP1.c`.
5. La depuración se hace con el MCU-Link integrado de la placa, con la configuración guardada en `TP1 LinkServer Debug.launch`.

Captura pendiente: **Importación del proyecto**
guardar como `img/nuevo-proyecto.png`

Figura 1: Importación del proyecto. Selección de la placa, del SDK, del ejemplo base y de los drivers incluidos.

2.3 Relojes

El reloj del núcleo determina cuántos ciclos hay disponibles para atender cada muestra, y el reloj de entrada del timer fija la resolución con la que se pueden generar las velocidades de muestreo. Se usa la configuración `BOARD_BootClockPLL150M`: el PLL0 toma el FRO de 48 MHz y lo lleva a

$$f_{\text{PLL0}} = \frac{48 \text{ MHz}}{N_{\text{DIV}}} \cdot \frac{M_{\text{DIV}}}{2} = \frac{48 \text{ MHz}}{8} \cdot \frac{50}{2} = 150 \text{ MHz}. \quad (1)$$

Con esa frecuencia se alimentan el núcleo y el CTIMER0. El ADC0 y el DAC0 toman el FRO_HF de 48 MHz, suficiente para ambos conversores.

Tabla 3: Frecuencias de reloj de los módulos utilizados.

Módulo	Fuente	Divisor	Frecuencia
Núcleo (main_clk)	PLL0	1	150 MHz
CTIMER0	PLL0	1	150 MHz
LPADC0	FRO_HF	1	48 MHz
DAC0	FRO_HF	1	48 MHz

Código 1: Configuración del PLL0 y de los selectores de reloj (board/clock_config.c).

```

1  /* Fragmento de board/clock_config.c (generado por Config Tools): PLL0 a 150 MHz y relojes de
   ↪ CTIMER0, ADC0 y DAC0 */
2  /*!< Set up PLL0 */
3  const pll_setup_t pll0Setup = {
4      .pllctrl = SCG_APLLCTRL_SOURCE(1U) | SCG_APLLCTRL_SEL1(27U) | SCG_APLLCTRL_SELP(13U),
5      .pllndiv = SCG_APLLNDIV_NDIV(8U),
6      .pllpdiv = SCG_APLLPDIV_PDIV(1U),
7      .pllmdiv = SCG_APLLMDIV_MDIV(50U),
8      .pllRate = 150000000U
9  };
10 CLOCK_SetPLLOFreq(&pll0Setup);                               /*!< Configure PLL0 to the desired
   ↪ values */
11 CLOCK_SetPll0MonitorMode(kSCG_Pll0MonitorDisable);          /* Pll0 Monitor is disabled */
12
13 /*!< Set up clock selectors */
14 CLOCK_AttachClk(kPLL0_to_MAIN_CLK);
15 CLOCK_AttachClk(kFRO_HF_to_ADC0);                            /*!< Switch ADC0 to FRO_HF */
16 CLOCK_AttachClk(kFRO_HF_to_ADC1);                            /*!< Switch ADC1 to FRO_HF */
17 CLOCK_AttachClk(kFRO_HF_to_DAC0);                            /*!< Switch DAC0 to FRO_HF */
18 CLOCK_AttachClk(kPLL0_to_CTIMER0);                          /*!< Switch CTIMER0 to PLL0 */
19
20 /*!< Set up dividers */
21 CLOCK_SetClkDiv(kCLOCK_DivAhbClk, 1U);                       /*!< Set AHBCLKDIV divider to value 1 */
22 CLOCK_SetClkDiv(kCLOCK_DivAdc0Clk, 1U);                     /*!< Set ADC0CLKDIV divider to value 1 */
23 CLOCK_SetClkDiv(kCLOCK_DivAdc1Clk, 1U);                     /*!< Set ADC1CLKDIV divider to value 1 */
24 CLOCK_SetClkDiv(kCLOCK_DivDac0Clk, 1U);                     /*!< Set DAC0CLKDIV divider to value 1 */
25 CLOCK_SetClkDiv(kCLOCK_DivCtimer0Clk, 1U);                  /*!< Set CTIMER0CLKDIV divider to value
   ↪ 1 */

```

Captura pendiente: **Config Tools — Clocks**
guardar como img/clocks.png

Figura 2: Config Tools — Clocks. Árbol de relojes: PLL0 a 150 MHz para el núcleo y el CTIMER0, FRO_HF para el ADC0 y el DAC0.

2.4 Pines

Tabla 4: Asignación de pines y su ubicación en los conectores de la placa.

Función	Pin	Periférico	Conector / observación
Entrada analógica	ADC0_A0	LPADC0 canal A0	J4[2] (Arduino A0), 0 V a 3,3 V
Salida analógica	P4_2	DAC0_OUT	J1[4]
Disparo del ADC	P0_19	CT0_MAT3	J2[13], punto de prueba
LED rojo	P0_10	GPIO0	Activo en bajo
LED verde	P0_27	GPIO0	Activo en bajo
LED azul	P1_2	GPIO1	Activo en bajo
Tecla SW2 (Run/Stop)	P0_23	GPIO0, canal IRQ 1	Flanco descendente
Tecla SW3 (velocidad)	P0_6	GPIO0, canal IRQ 0	Flanco descendente, pull-up interno
Consola de depuración	P1_8, P1_9	LP_FLEXCOMM4	MCU-Link

La salida del Match 3 del CTIMER0 se saca además por P0_19: esa señal es la que dispara al ADC, así que con el osciloscopio en J2[13] se mide directamente la frecuencia de muestreo (sección 5).



Figura 3: Config Tools — Pins. Ruteo de pines: ADC0_A0, DAC0_OUT, CT0_MAT3, LED RGB y teclas.

2.5 Periféricos

2.5.1 Timer de muestreo (CTIMER0)

El CTIMER0 cuenta a 150 MHz (prescaler 0) y usa el canal de *match* 3 con reinicio del contador y salida en modo *toggle*. Cada vez que la cuenta llega a MR la salida cambia de estado, así que en P0_19 aparece una onda cuadrada de período $2(MR+1)$ ciclos del timer. Esa señal se conecta por el **INPUTMUX** a la entrada de disparo 0 del LPADC0 (`kINPUTMUX_Ctimer0M3ToAdc0Trigger`), que arranca una conversión por período. De esta forma el instante de muestreo lo fija el hardware y no depende de la latencia de las interrupciones: el CTIMER no genera interrupciones.

Tabla 5: Configuración del CTIMER0.

Parámetro	Valor
Modo	Timer (<code>kCTIMER_TimerMode</code>)
Reloj / prescaler	150 MHz / 0
Canal	Match 3
Acción en el match	Reinicio del contador, sin detener, sin interrupción
Salida	Toggle (<code>kCTIMER_Output_Toggle</code>), P0_19
Valor inicial de MR	9374 (generado por Config Tools)
Destino del disparo	INPUTMUX → LPADC0, trigger 0

Código 2: Inicialización del CTIMER0 (board/peripherals.c).

```

1  /* Fragmento de board/peripherals.c (generado por Config Tools): CTIMER0 */
2  const ctimer_config_t CTIMER0_config = {
3      .mode = kCTIMER_TimerMode,
4      .input = kCTIMER_Capture_0,
5      .prescale = 0
6  };
7  const ctimer_match_config_t CTIMER0_Match_3_config = {
8      .matchValue = 9374,
9      .enableCounterReset = true,
10     .enableCounterStop = false,
11     .outControl = kCTIMER_Output_Toggle,
12     .outPinInitState = false,
13     .enableInterrupt = false
14 };
15
16 static void CTIMER0_init(void) {
17     /* CTIMER0 peripheral initialization */
18     CTIMER_Init(CTIMER0_PERIPHERAL, &CTIMER0_config);
19     /* Match channel 3 of CTIMER0 peripheral initialization */
20     CTIMER_SetupMatch(CTIMER0_PERIPHERAL, CTIMER0_MATCH_3_CHANNEL, &CTIMER0_Match_3_config);
21     /* Start the timer */
22     CTIMER_StartTimer(CTIMER0_PERIPHERAL);
23
24 }

```

Código 3: Ruteo del Match 3 al disparo del ADC0 y salida del DAC (board/pin_mux.c).

```

1  /* Fragmento de board/pin_mux.c (generado por Config Tools): ruteo del disparo y salida del
   ⇨ DAC */
2  /* Timer CTIMER0 Match 3 is selected as trigger input for ADC0 channel 0 */
3  INPUTMUX_AttachSignal(INPUTMUX0, 0U, kINPUTMUX_Ctimer0M3ToAdc0Trigger);
4
5  /* PORT0_19 (pin C9) is configured as CT0_MAT3 */
6  PORT_SetPinMux(PORT0, 19U, kPORT_MuxAlt4);
7
8  /* PORT4_2 (pin T1) is configured as DAC0_OUT */
9  PORT_SetPinMux(PORT4, 2U, kPORT_MuxAlt0);

```

Captura pendiente: **Config Tools — CTIMER0**
guardar como img/peripherals-ctimer.png

Figura 4: Config Tools — CTIMER0. Modo timer, prescaler, canal de match 3 con reinicio y salida en toggle.

2.5.2 Conversor A/D (LPADC0)

El ADC0 se usa en resolución estándar (12 bit, single-ended) sobre el canal A0. El trigger 0, habilitado por hardware, ejecuta el comando 1, que convierte una vez y deja el resultado en la FIFO0. Al completarse el trigger se genera la interrupción ADC0_IRQn. Antes de habilitar la interrupción, main() ejecuta la calibración de *offset* y la autocalibración, y vacía la FIFO.

Tabla 6: Configuración del LPADC0.

Parámetro	Valor
Reloj	FRO_HF, 48 MHz (asíncrono)
Resolución	Estándar: 12 bit single-ended
Canal	A0, lado A (kLPADC_SampleChannelSingleEndSideA)
Tiempo de muestreo	3 ciclos de ADCK (kLPADC_SampleTimeADCK3)
Promediado	Ninguno (1 conversión por disparo)
Referencia	kLPADC_ReferenceVoltageAlt1
Trigger	Trigger 0 → comando 1, por hardware
FIFO	FIFO0, marca de agua 0
Interrupción	Fin de trigger 0 (kLPADC_Trigger0CompletionInterruptEnable)
Calibración	<i>Offset</i> + autocalibración en main()

El VREF0 se enciende en modo *bandgap* sólo para dar la corriente de polarización que necesita el LPADC.

Código 4: Inicialización del LPADC0 (board/peripherals.c).

```

1  /* Fragmento de board/peripherals.c (generado por Config Tools): LPADC0 */
2  const lpadc_config_t ADC0_config = {
3      .enableInDozeMode = true,
4      .conversionAverageMode = kLPADC_ConversionAverage1,
5      .enableAnalogPreliminary = false,
6      .powerUpDelay = 0x80UL,
7      .referenceVoltageSource = kLPADC_ReferenceVoltageAlt1,
8      .powerLevelMode = kLPADC_PowerLevelAlt1,
9      .triggerPriorityPolicy = kLPADC_ConvPreemptImmediatelyNotAutoResumed,
10     .enableConvPause = false,
11     .convPauseDelay = 0UL,
12     .FIFO0Watermark = 0UL,
13     .FIFO1Watermark = 0UL
14 };
15 lpadc_conv_command_config_t ADC0_commandsConfig[1] = {
16     {
17         .sampleChannelMode = kLPADC_SampleChannelSingleEndSideA,
18         .channelNumber = 0U,
19         .channelBNumber = 0U,
20         .chainedNextCommandNumber = 0,
21         .enableChannelB = false,
22         .enableAutoChannelIncrement = false,
23         .loopCount = 0UL,
24         .hardwareAverageMode = kLPADC_HardwareAverageCount1,
25         .sampleTimeMode = kLPADC_SampleTimeADCK3,
26         .hardwareCompareMode = kLPADC_HardwareCompareDisabled,
27         .hardwareCompareValueHigh = 0UL,
28         .hardwareCompareValueLow = 0UL,
29         .conversionResolutionMode = kLPADC_ConversionResolutionStandard,
30         .enableWaitTrigger = true
31     }
32 };
33 lpadc_conv_trigger_config_t ADC0_triggersConfig[1] = {
34     {
35         .targetCommandId = 1,
36         .delayPower = 0UL,
37         .channelAFIFOSelect = 0,
38         .channelBFIFOSelect = 0,
39         .priority = 1,
40         .enableHardwareTrigger = true

```

```

41 }
42 };
43
44 static void ADC0_init(void) {
45     /* Initialize LPADC converter */
46     LPADC_Init(ADC0_PERIPHERAL, &ADC0_config);
47     /* Configure conversion command 1. */
48     LPADC_SetConvCommandConfig(ADC0_PERIPHERAL, 1, &ADC0_commandsConfig[0]);
49     /* Configure trigger 0. */
50     LPADC_SetConvTriggerConfig(ADC0_PERIPHERAL, 0, &ADC0_triggersConfig[0]);
51     /* Enable interrupts from LPADC */
52     LPADC_EnableInterrupts(ADC0_PERIPHERAL, (kLPADC_Trigger0CompletionInterruptEnable));
53     /* Enable interrupt ADC0_IRQN request in the NVIC */
54     EnableIRQ(ADC0_IRQN);
55 }
56 }

```

Captura pendiente: **Config Tools — LPADC0**
guardar como `img/peripherals-lpadc.png`

Figura 5: Config Tools — LPADC0. Comando de conversión, trigger por hardware e interrupción.

2.5.3 Conversor D/A (DAC0)

Se usa el LPDAC0 de 12 bit sin FIFO: cada llamada a `DAC_SetData()` actualiza la salida directamente. La referencia es `VREFH` (`kDAC_ReferenceVoltageSourceAlt3`) y la salida pasa por el amplificador *buffer* interno. El módulo se enciende desde el SPC antes de inicializarlo.

Tabla 7: Configuración del DAC0.

Parámetro	Valor
Resolución	12 bit, dato justificado a derecha (0 a 4095)
FIFO	Deshabilitada (escritura directa)
Referencia de tensión	VREFH (<code>kDAC_ReferenceVoltageSourceAlt3</code>)
Referencia de corriente	PTAT interna
Buffer de salida	Habilitado
Salida	DAC0_OUT, P4_2

Código 5: Inicialización del DAC0 y del VREF0 (`board/peripherals.c`).

```

1  /* Fragmento de board/peripherals.c (generado por Config Tools): DAC0 y VREF0 */
2  const dac_config_t DAC0_config = {
3      .fifoWatermarkLevel = OUL,
4      .fifoTriggerMode = kDAC_FIFOTriggerByHardwareMode,
5      .fifoWorkMode = kDAC_FIFODisabled,
6      .referenceVoltageSource = kDAC_ReferenceVoltageSourceAlt3,
7      .referenceCurrentSource = kDAC_ReferenceCurrentSourcePtat,
8      .enableOpampBuffer = true,
9      .periodicTriggerNumber = OUL,

```

```

10 .periodicTriggerWidth = OUL,
11 .syncTime = 1UL,
12 .enableLowerLowPowerMode = false,
13 };
14
15 static void DAC0_init(void) {
16     /* Power up analog module in SPC */
17     SPC_EnableActiveModeAnalogModules(SPC0, kSPC_controlDac0);
18     /* Initialize the LPDAC */
19     DAC_Init(DACO_PERIPHERAL, &DAC0_config);
20     /* Enable the LPDAC */
21     DAC_Enable(DACO_PERIPHERAL, true);
22 }
23
24
25 static void VREF0_init(void) {
26     /* Power up VREF */
27     vref_config_t vrefConfig;
28
29     /* enable VREF */
30     SPC_EnableActiveModeAnalogModules(SPC0, kSPC_controlVref);
31
32     VREF_GetDefaultConfig(&vrefConfig);
33     vrefConfig.bufferMode = kVREF_ModeBandgapOnly;
34     /* Initialize VREF module, the VREF module is only used to supply the bias current for
35     ↪ LPADC. */
36     VREF_Init(VREF0_PERIPHERAL, &vrefConfig);
37 }

```

Captura pendiente: **Config Tools — DAC0**
guardar como `img/peripherals-dac.png`

Figura 6: Config Tools — DAC0. Configuración del DAC de 12 bits.

2.5.4 Teclas y LED (GPIO)

Los tres segmentos del LED RGB se configuran como salidas GPIO; son activos en bajo, así que escribir 0 enciende el color. Las teclas se configuran como entradas con interrupción por flanco descendente (se presionan a masa). El GPIO0 tiene dos líneas de interrupción: SW3 (P0_6) usa la línea 0 (GPIO00_IRQn) y SW2 (P0_23) se asigna a la línea 1 (GPIO01_IRQn) mediante el campo IRQS del registro ICR[23]. Así cada tecla tiene su propia rutina de atención.

Código 6: Interrupción de las teclas (board/pin_mux.c).

```

1  /* Fragmento de board/pin_mux.c (generado por Config Tools): interrupcion de las teclas */
2  GPIO0->ICR[23] = ((GPIO0->ICR[23] &
3      /* Mask bits to zero which are setting */
4      ~(GPIO_ICR_IRQS_MASK | GPIO_ICR_ISF_MASK)))
5
6      /* Interrupt Select: Interrupt, trigger output, or DMA request 1. */
7      | GPIO_ICR_IRQS(ICR_IRQS_irqs1));
8
9  /* Interrupt configuration on GPIO0_23 (pin B7): Interrupt on falling edge */

```

```

10 GPIO_SetPinInterruptConfig(BOARD_INITBUTTONSPINS_SW2_GPIO, BOARD_INITBUTTONSPINS_SW2_PIN,
    ↪ kGPIO_InterruptFallingEdge);
11
12 /* Interrupt configuration on GPIO0_6 (pin C14): Interrupt on falling edge */
13 GPIO_SetPinInterruptConfig(BOARD_INITBUTTONSPINS_SW3_GPIO, BOARD_INITBUTTONSPINS_SW3_PIN,
    ↪ kGPIO_InterruptFallingEdge);

```

Captura pendiente: **Config Tools — GPIO0**
guardar como `img/peripherals-gpio.png`

Figura 7: Config Tools — GPIO0. Interrupciones de las teclas en las líneas 0 y 1 del GPIO0.

2.5.5 Prioridades de interrupción

Tabla 8: Interrupciones del sistema. No se modificó la prioridad por defecto, así que las tres tienen prioridad 0 (en el Cortex-M33 un número menor es mayor prioridad).

Fuente	Handler	Prioridad
LPADC0 (fin de conversión)	ADC0_IRQHandler	0
Tecla SW3 (velocidad)	GPIO00_IRQHandler	0
Tecla SW2 (Run/Stop)	GPIO01_IRQHandler	0

Con la misma prioridad ninguna interrupción desaloja a otra: si se presiona una tecla justo cuando termina una conversión, la segunda rutina espera a que termine la primera. Como ambas son cortas frente al período de muestreo más chico (20,8 μs, unos 3100 ciclos del núcleo), la demora no provoca pérdida de muestras. Además el disparo del ADC no depende del CPU, así que esa espera no altera el instante de muestreo.

3 Diseño de la aplicación

3.1 Arquitectura general

La figura 8 muestra las partes de la aplicación. El camino de la señal (fila superior) funciona por completo a partir de hardware e interrupciones: el CTIMER0 dispara cada conversión a través del INPUTMUX, el LPADC0 deja el resultado en su FIFO y la rutina de interrupción lo guarda en el buffer circular y lo escribe en el DAC0. Las teclas sólo modifican el estado (índice de velocidad y modo Run/Stop): la de velocidad reprograma el match del CTIMER0 y el color del LED, y la de Run/Stop cambia lo que hace la rutina del ADC con cada muestra. El lazo principal queda vacío.

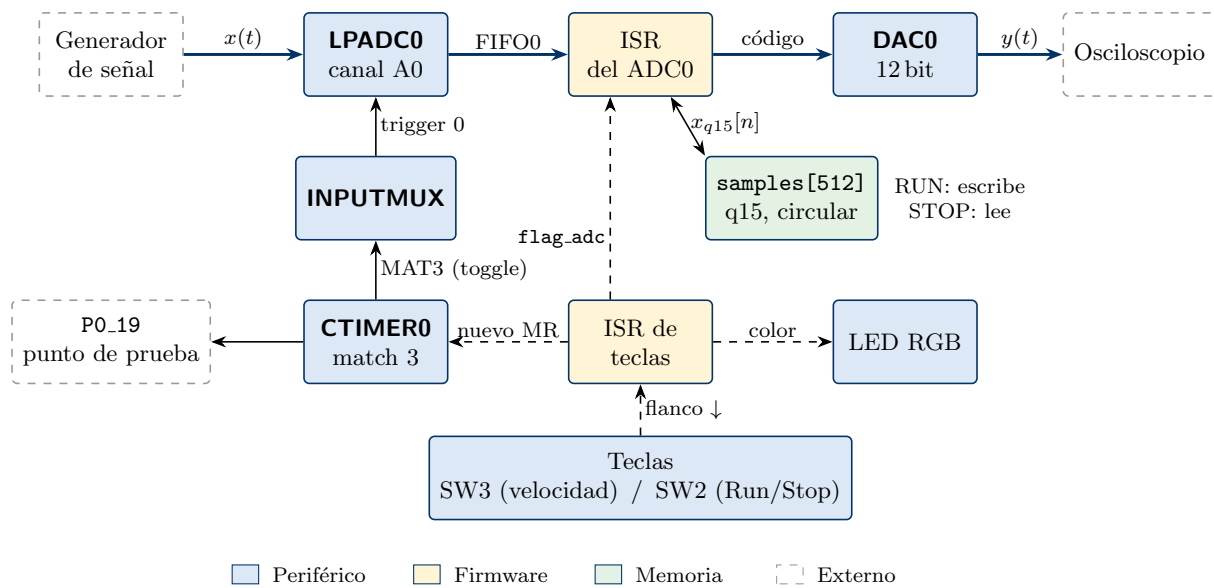


Figura 8: Diagrama de bloques de la aplicación. Trazo continuo azul: señal; negro: datos y disparo; punteado: control.

3.2 Generación de las velocidades de muestreo

Con el timer contando a f_{timer} , reiniciándose en el match y con la salida en toggle, un período completo de la señal de disparo abarca dos matches, es decir $2(\text{MR} + 1)$ ciclos del timer:

$$f_s = \frac{f_{\text{timer}}}{2(\text{MR} + 1)}. \quad (2)$$

En el firmware los valores de MR están en el arreglo `sample_vel` (constantes `VEL_0` a `VEL_4`) y se calcularon como $\text{MR} = \lfloor f_{\text{timer}} / (2f_s) \rfloor$. La tabla 9 aplica la ecuación (2) con $f_{\text{timer}} = 150 \text{ MHz}$ a esos valores. El índice de estado coincide con la posición en el arreglo.

Tabla 9: Velocidades de muestreo, valor de match programado y color del LED.

Estado	f_s nominal	MR	f_s real	Error	LED
E0	8 000 Hz	9 375	7 999,15 Hz	−0,011 %	● Verde
E1	16 000 Hz	4 687	15 998,29 Hz	−0,011 %	● Amarillo
E2	22 000 Hz	3 409	21 994,13 Hz	−0,027 %	● Azul
E3	44 000 Hz	1 704	43 988,27 Hz	−0,027 %	● Rojo
E4	48 000 Hz	1 562	47 984,64 Hz	−0,032 %	○ Blanco

El error se debe a que la ecuación (2) tiene $MR+1$ en el denominador: con $MR = f_{\text{timer}}/(2f_s) - 1$ (por ejemplo 9374 para 8 kS/s, que es el valor que genera Config Tools para el arranque) las velocidades de 8, 16 y 48 kS/s serían exactas. En cualquier caso el error es menor a 0,04 %, despreciable para esta aplicación.

Interpretación de 22K/S y 44K/S

Se tomaron literalmente como 22 kS/s y 44 kS/s. Si la cátedra se refiere a las frecuencias estándar de audio (22,05 kHz y 44,1 kHz), basta cambiar VEL_2 y VEL_3 en `source/TP1.c` por 3401 y 1700, y actualizar la tabla en `main.tex`.

3.3 Máquina de estados

El comportamiento de las teclas se modela con dos variables de estado independientes (figura 9):

- **state_counter**: índice de velocidad. Avanza en forma circular con SW3; al llegar a E4 vuelve a E0. Cada cambio reprograma el match del CTIMER0 y el color del LED.
- **flag_adc**: modo Run/Stop. Alterna con SW2. En RUN cada muestra se guarda en el buffer y se envía al DAC. En STOP el buffer queda congelado con las últimas 512 muestras y el DAC reproduce su contenido en forma cíclica, a la misma velocidad de muestreo.

Las dos teclas actúan en cualquier estado: se puede cambiar la velocidad estando en STOP (cambia la velocidad de reproducción). El LED sólo indica la velocidad, no el modo Run/Stop. Al reiniciar la placa el sistema arranca en E0 y en RUN.

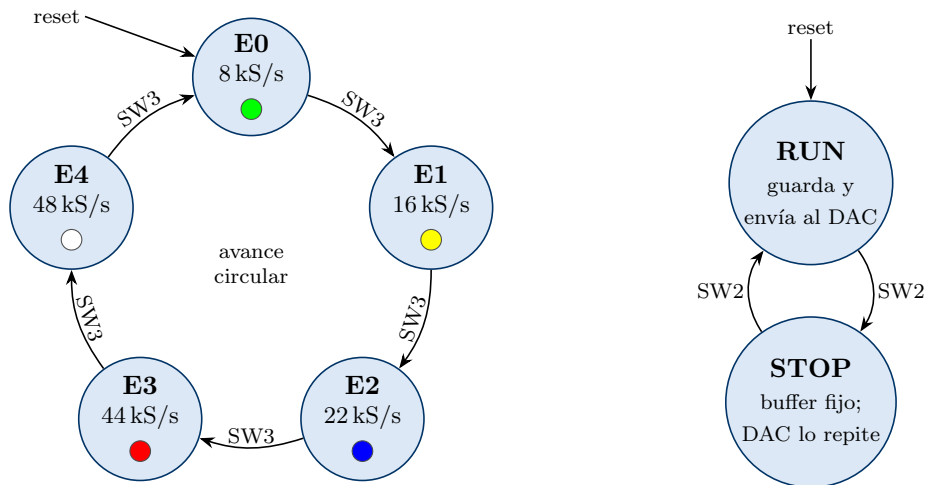


Figura 9: Izquierda: selección circular de la velocidad de muestreo con SW3 y color del LED en cada estado. Derecha: modo de adquisición con SW2.

3.4 Formato q15 y conversiones

Un número q15 es un entero de 16 bits con signo que representa el valor

$$v = \frac{x_{q15}}{2^{15}} \in [-1, 1 - 2^{-15}]. \quad (3)$$

En resolución estándar el LPADC entrega el resultado de 12 bit alineado a la izquierda dentro de la palabra de 16 bits del registro de la FIFO. El firmware guarda esa palabra sin modificar en el buffer como `q15_t`, de modo que la muestra queda como una fracción de la escala completa. Como la entrada es single-ended, el valor es siempre no negativo.

Para el DAC hay que volver a un código de 12 bits justificado a derecha. En RUN el firmware lo obtiene del resultado del ADC con un corrimiento y una máscara:

$$\text{código}_{\text{DAC}} = (x_{\text{ADC}} \gg 4) \& 0\text{x0FFF}. \quad (4)$$

[Verificar con el osciloscopio que la amplitud de salida sea igual a la de entrada: según el ejemplo del SDK el resultado de 12 bits está corrido 3 lugares (no 4); con 4 la salida tendría la mitad de amplitud.]

En STOP el valor que se envía al DAC es la muestra guardada `samples[dac_count]`. [La muestra guardada es la palabra de 16 bits del ADC: aplicarle el mismo corrimiento antes de `DAC_SetData()` para que la reproducción tenga la escala correcta.]

3.5 Rutinas de interrupción

Toda la aplicación corre en tres rutinas de interrupción (figura 10). La del ADC se ejecuta una vez por muestra; las de las teclas, una vez por pulsación.

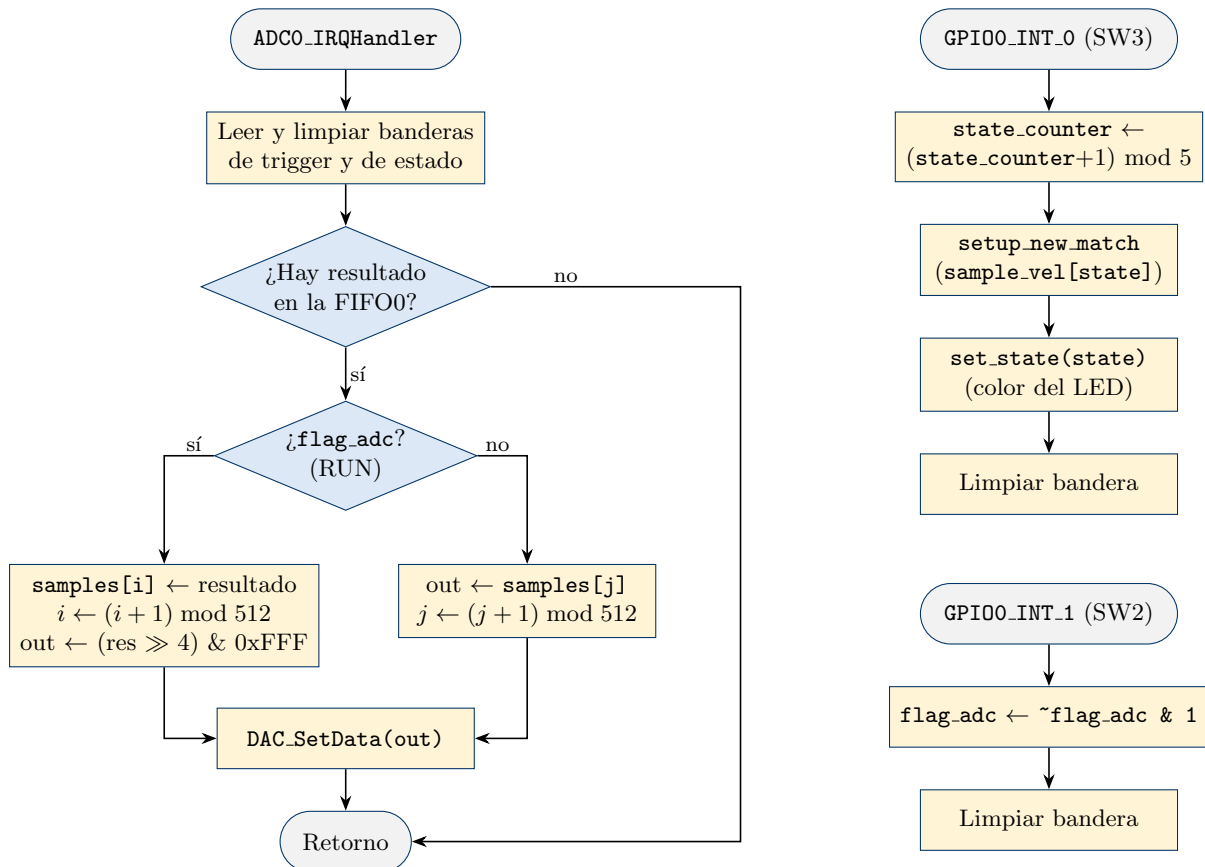


Figura 10: Diagramas de flujo de las rutinas de interrupción. Izquierda: fin de conversión del ADC. Derecha: teclas de velocidad (SW3) y de Run/Stop (SW2).

4 Implementación

Toda la lógica propia está en `source/TP1.c`; el resto del proyecto es código generado por Config Tools (sección 2) y drivers del SDK. El archivo completo está en el Anexo A.

4.1 Constantes y variables globales

Los valores de match de cada velocidad se agrupan en `sample_vel`, así que avanzar de estado es sólo avanzar un índice. Las variables que comparten las rutinas de interrupción se declaran `volatile`. El buffer `samples` es un arreglo de 512 elementos `q15_t`; su índice de escritura (`sample_count`) y el de lectura en STOP (`dac_count`) son variables estáticas de la rutina del ADC.

Código 7: Definiciones y variables globales.

```

1  /* Fragmento de source/TP1.c: constantes, variables globales y buffer circular */
2  #define MAX_FREQ CLOCK_GetCTimerClkFreq(0)
3
4  #define VEL_0      9375
5  #define VEL_1      4687
6  #define VEL_2      3409
7  #define VEL_3      1704
8  #define VEL_4      1562
9
10 #define LED_RED_PIN BOARD_INITLEDSPINS_LED_RED_PIN
11 #define LED_GREEN_PIN BOARD_INITLEDSPINS_LED_GREEN_PIN
12 #define LED_BLUE_PIN BOARD_INITLEDSPINS_LED_BLUE_PIN
13
14 #define SAMPLES_NUM 512
15
16 volatile int state_counter = 0;
17 volatile int flag_adc = 1;
18
19 volatile uint32_t last_adc = 0;  /* global */
20 volatile uint32_t last_dac = 0;  /* global */
21
22 uint32_t sample_vel[5] = {VEL_0, VEL_1, VEL_2, VEL_3, VEL_4};
23 volatile q15_t samples[SAMPLES_NUM];

```

4.2 Programa principal

`main()` inicializa pines, relojes y periféricos con las funciones generadas por Config Tools, fija el color del estado inicial y calibra el ADC con la interrupción deshabilitada y el timer detenido, para que no lleguen disparos durante la calibración. Luego vacía la FIFO, habilita la interrupción, arranca el CTIMER0 y queda en un lazo vacío: a partir de ese momento todo ocurre en las interrupciones.

Código 8: Programa principal.

```

1  /* Fragmento de source/TP1.c: programa principal */
2  int main(void) {
3
4      /* Init board hardware. */
5      BOARD_InitBootPins();
6      BOARD_InitBootClocks();
7      BOARD_InitBootPeripherals();
8      BOARD_InitLEDsPins();
9      BOARD_InitBUTTONsPins();

```

```

10 #ifndef BOARD_INIT_DEBUG_CONSOLE_PERIPHERAL
11     /* Init FSL debug console. */
12     BOARD_InitDebugConsole();
13 #endif
14
15     set_state(0);
16     DisableIRQ(ADCO_IRQn);
17     CTIMER_StopTimer(CTIMER0);
18     LPADC_DoOffsetCalibration(ADCO);
19     LPADC_DoAutoCalibration(ADCO);
20     LPADC_DoResetFIFO0(ADCO);
21     EnableIRQ(ADCO_IRQn);
22     CTIMER_StartTimer(CTIMER0);
23
24 //     /* Enter an infinite loop, just incrementing a counter. */
25     while(1) {
26     }
27     return 0 ;
28 }

```

4.3 Cambio de velocidad

La función `setup_new_match()` detiene el CTIMER0, carga el nuevo valor de match con la misma configuración que generó Config Tools (reinicio en el match, salida en toggle), reinicia la cuenta y vuelve a arrancar el timer. Reiniciar la cuenta evita que, si el nuevo MR es menor que la cuenta actual, el contador tenga que dar la vuelta completa de 32 bits antes del próximo match.

Código 9: Reprogramación del CTIMER0.

```

1  /* Fragmento de source/TP1.c: reprogramacion del CTIMER0 al cambiar de velocidad */
2  void setup_new_match(int m_val){
3
4      uint32_t match_val = m_val;
5      CTIMER_StopTimer(CTIMER0_PERIPHERAL);
6      const ctimer_match_config_t CTIMER0_Match_3_config = {
7          .matchValue = match_val,
8          .enableCounterReset = true,
9          .enableCounterStop = false,
10         .outControl = kCTIMER_Output_Toggle,
11         .outPinInitState = false,
12         .enableInterrupt = false
13     };
14     CTIMER_SetupMatch(CTIMER0_PERIPHERAL, CTIMER0_MATCH_3_CHANNEL, &CTIMER0_Match_3_config);
15     CTIMER_Reset(CTIMER0_PERIPHERAL);
16     CTIMER_StartTimer(CTIMER0_PERIPHERAL);
17 }

```

`set_state()` traduce el índice de velocidad a los tres pines del LED. Como el LED es activo en bajo, un 0 enciende el segmento.

Código 10: Color del LED según la velocidad.

```

1  /* Fragmento de source/TP1.c: color del LED RGB segun la velocidad (LED activo en bajo) */
2  void set_state(int state_number){
3      switch (state_number) {
4          case 0:
5              GPIO_PinWrite(GPIO0, LED_RED_PIN, 1);
6              GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
7              GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);

```

```

8         break;
9     case 1:
10        GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
11        GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
12        GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);
13        break;
14     case 2:
15        GPIO_PinWrite(GPIO0, LED_RED_PIN, 1);
16        GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 1);
17        GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 0);
18        break;
19     case 3:
20        GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
21        GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 1);
22        GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);
23        break;
24     default:
25        GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
26        GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
27        GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 0);
28        break;
29 }
30 }

```

4.4 Rutinas de interrupción

La rutina de SW3 avanza el índice en forma circular (módulo del tamaño del arreglo), reprograma el timer y actualiza el LED. La de SW2 alterna `flag_adc` entre 0 y 1.

Código 11: Rutinas de interrupción de las teclas.

```

1  /* Fragmento de source/TP1.c: interrupciones de las teclas SW3 (canal 0) y SW2 (canal 1) */
2  void GPIO0_INT_0_IRQHANDLER(void) {
3      /* Get pin flags 0 */
4      uint32_t pin_flags0 = GPIO_GpioGetInterruptChannelFlags(GPIO0, 0U);
5
6      /* Place your interrupt code here */
7      state_counter = (state_counter + 1) % (sizeof(sample_vel) / sizeof(sample_vel[0]));
8      setup_new_match(sample_vel[state_counter]);
9      set_state(state_counter); /* Clear pin flags 0 */
10     GPIO_GpioClearInterruptChannelFlags(GPIO0, pin_flags0, 0U);
11
12     /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
13     Store immediate overlapping exception return operation might vector to incorrect
14     ↳ interrupt. */
15     #if defined __CORTEX_M && (__CORTEX_M == 4U)
16         __DSB();
17     #endif
18 }
19
20 void GPIO0_INT_1_IRQHANDLER(void) {
21     /* Get pin flags 1 */
22     uint32_t pin_flags1 = GPIO_GpioGetInterruptChannelFlags(GPIO0, 1U);
23
24     /* Place your interrupt code here */
25     flag_adc = ~flag_adc & 1;
26     /* Clear pin flags 1 */
27     GPIO_GpioClearInterruptChannelFlags(GPIO0, pin_flags1, 1U);
28
29     /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
30     Store immediate overlapping exception return operation might vector to incorrect

```

```

    ↪ interrupt. */
30  #if defined __CORTEX_M && (__CORTEX_M == 4U)
31      __DSB();
32  #endif
33  }

```

La rutina del ADC limpia las banderas, lee el resultado de la FIFO0 y, según el modo, guarda la muestra y la envía al DAC (RUN) o envía al DAC la muestra guardada siguiente (STOP). En ambos casos el índice avanza módulo 512, que es lo que hace circular al buffer.

Código 12: Rutina de interrupción del ADC0.

```

1  /* Fragmento de source/TP1.c: interrupcion de fin de conversion del ADC0 */
2  void ADC0_IRQHANDLER(void) {
3      uint32_t trigger_status_flag;
4      uint32_t status_flag;
5      /* Trigger interrupt flags */
6      trigger_status_flag = LPADC_GetTriggerStatusFlags(ADC0_PERIPHERAL);
7      /* Interrupt flags */
8      status_flag = LPADC_GetStatusFlags(ADC0_PERIPHERAL);
9      /* Clears trigger interrupt flags */
10     LPADC_ClearTriggerStatusFlags(ADC0_PERIPHERAL, trigger_status_flag);
11     /* Clears interrupt flags */
12     LPADC_ClearStatusFlags(ADC0_PERIPHERAL, status_flag);
13
14     /* Place your code here */
15     static uint16_t sample_count = 0;
16     static uint16_t dac_count = 0;
17     lpadc_conv_result_t result;
18
19     if (LPADC_GetConvResult(ADC0_PERIPHERAL, &result, 0)) {
20         uint16_t v = (result.convValue >> 4) & 0xFFFF; /* solo si single-ended 12 bit */
21         uint16_t out;
22         if (flag_adc) { samples[sample_count] = (q15_t)result.convValue; sample_count = (
23             ↪ sample_count + 1) % SAMPLES_NUM; out = v; }
24         else { out = (uint16_t)samples[dac_count]; dac_count = (dac_count + 1) %
25             ↪ SAMPLES_NUM; }
26         DAC_SetData(DAC0, out);
27     }
28
29     /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
30        Store immediate overlapping exception return operation might vector to incorrect
31        ↪ interrupt. */
32     #if defined __CORTEX_M && (__CORTEX_M == 4U)
33         __DSB();
34     #endif
35 }

```

5 Verificación

5.1 Procedimiento

1. Conectar el generador a J4[2] (ADC0_A0) con una senoidal de [frecuencia], amplitud [Vpp] y un *offset* que la mantenga entre 0 y 3,3 V.
2. Canal 1 del osciloscopio en la entrada, canal 2 en J1[4] (DAC0_OUT).
3. Recorrer las cinco velocidades con SW3, verificar el color del LED y medir en J2[13] (P0_19) la frecuencia de la señal de disparo, que es igual a f_s .
4. Presionar SW2 y verificar que la salida pasa a repetir el contenido del buffer aunque cambie la entrada; volver a presionar para reanudar.
5. Detener el programa con el depurador y observar el arreglo `samples` para comprobar que contiene 512 muestras de la señal.

5.2 Frecuencia de muestreo medida

Tabla 10: Frecuencia de la señal de disparo medida en P0_19.

Estado	f_s esperada [Hz]	f_s medida [Hz]	LED observado
E0	7999,15		
E1	15998,29		
E2	21994,13		
E3	43988,27		
E4	47984,64		

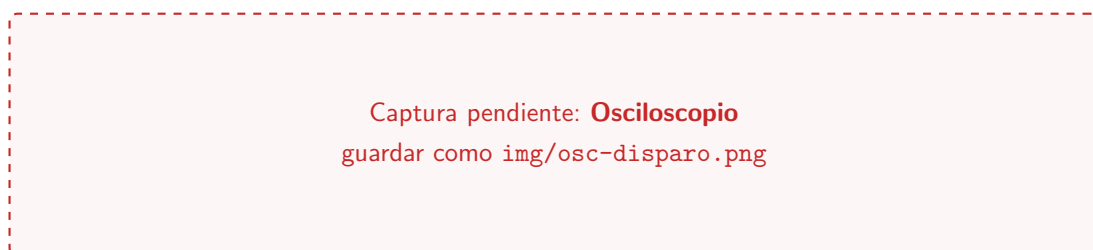


Figura 11: Osciloscopio. Señal de disparo del ADC en P0_19 (CT0_MAT3).

5.3 Señal de entrada y salida

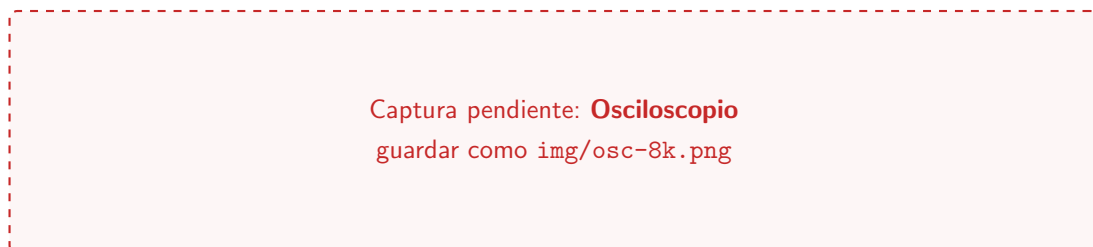


Figura 12: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) a 8kS/s.

Captura pendiente: **Osciloscopio**
guardar como `img/osc-48k.png`

Figura 13: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) a 48 kS/s.

Captura pendiente: **Osciloscopio**
guardar como `img/osc-stop.png`

Figura 14: Osciloscopio. Modo STOP: la salida repite el contenido del buffer.

Captura pendiente: **Depurador**
guardar como `img/debug-buffer.png`

Figura 15: Depurador. Contenido del arreglo `samples` con el programa detenido.

5.4 Cumplimiento de los requerimientos

Tabla 11: Verificación de los requerimientos de la consigna.

Req.	Cómo se implementa	Evidencia	Resultado
R1	CTIMER0 + INPUTMUX disparan el LPADC0 a la f_s elegida	Tabla 10	☐
R2	SW3 avanza <code>state_counter</code> módulo 5	Demostración	☐
R3	<code>set_state()</code> : un color por velocidad	Tabla 9	☐
R4	SW2 alterna <code>flag_adc</code>	Figura 14	☐
R5	<code>samples[512]</code> de <code>q15_t</code> , índice módulo 512	Figura 15	☐
R6	<code>DAC_SetData()</code> en cada interrupción del ADC	Figuras 12 y 13	☐

6 Conclusiones

Se implementó sobre la FRDM-MCXN947 una aplicación que digitaliza una señal con el LPADC0 a cinco velocidades seleccionables, guarda las muestras en un buffer circular de 512 valores `q15_t` y las reproduce por el DAC0 de 12 bits.

La decisión principal del diseño fue disparar el ADC por hardware: la salida del match del CTIMER0 llega al LPADC por el INPUTMUX, sin pasar por el CPU. Así el período de muestreo depende sólo del reloj del timer y no de la latencia de las interrupciones, y el procesador interviene únicamente para mover cada resultado al buffer y al DAC. Con el timer a 150 MHz todas las velocidades se obtienen con un error menor a 0,04 %, y a 48 kS/s quedan unos 3100 ciclos del núcleo por muestra, de los cuales la rutina del ADC usa una fracción pequeña.

Tener la señal de disparo en un pin (`P0_19`) resultó útil para verificar: la frecuencia de muestreo se mide directamente con el osciloscopio, sin depender del depurador.

[Agregar observaciones propias a partir de las mediciones: forma escalonada de la salida según f_s , efecto de muestrear señales cercanas a $f_s/2$, necesidad de un filtro de reconstrucción a la salida del DAC.]

6.1 Mejoras posibles

- Agregar antirrebote a las teclas: con interrupción por flanco, un rebote puede avanzar más de un estado por pulsación.
- Dar mayor prioridad a la interrupción del ADC que a las de las teclas.
- Indicar el modo STOP en el LED (por ejemplo, con parpadeo).
- Usar $MR = f_{\text{timer}}/(2f_s) - 1$ para que las velocidades que dividen exactamente a 150 MHz no tengan error.

A Código fuente

El proyecto completo de MCUXpresso (incluidos los archivos generados por Config Tools, los drivers del SDK y CMSIS-DSP) está en la carpeta TP1/ del repositorio <https://github.com/Mati-Costamagna/procesamiento-digital-de-senales>. A continuación se transcribe el archivo de la aplicación.

Código 13: source/TP1.c completo.

```

1  /*
2   * Copyright 2016-2026 NXP
3   * All rights reserved.
4   *
5   * SPDX-License-Identifier: BSD-3-Clause
6   */
7
8  /**
9   * @file      TP1.c
10  * @brief     Application entry point.
11  */
12 #include <stdio.h>
13 #include "board.h"
14 #include "peripherals.h"
15 #include "pin_mux.h"
16 #include "clock_config.h"
17 #include "fsl_debug_console.h"
18 #include "arm_math.h"
19 #include "fsl_ctimer.h"
20 #include "fsl_lpadc.h"
21 #include "fsl_dac.h"
22 /* TODO: insert other include files here. */
23
24 /* TODO: insert other definitions and declarations here. */
25 #define MAX_FREQ CLOCK_GetCTimerClkFreq(0)
26
27 #define VEL_0      9375
28 #define VEL_1      4687
29 #define VEL_2      3409
30 #define VEL_3      1704
31 #define VEL_4      1562
32
33 #define LED_RED_PIN BOARD_INITLEDSPINS_LED_RED_PIN
34 #define LED_GREEN_PIN BOARD_INITLEDSPINS_LED_GREEN_PIN
35 #define LED_BLUE_PIN BOARD_INITLEDSPINS_LED_BLUE_PIN
36
37 #define SAMPLES_NUM 512
38
39 volatile int state_counter = 0;
40 volatile int flag_adc = 1;
41
42 volatile uint32_t last_adc = 0; /* global */
43 volatile uint32_t last_dac = 0; /* global */
44
45 uint32_t sample_vel[5] = {VEL_0, VEL_1, VEL_2, VEL_3, VEL_4};
46 volatile q15_t samples[SAMPLES_NUM];
47
48 void setup_new_match(int m_val){
49
50     uint32_t match_val = m_val;
51     CTIMER_StopTimer(CTIMERO_PERIPHERAL);
52     const ctimer_match_config_t CTIMERO_Match_3_config = {
53         .matchValue = match_val,
54         .enableCounterReset = true,

```

```

55     .enableCounterStop = false,
56     .outControl = kCTIMER_Output_Toggle,
57     .outPinInitState = false,
58     .enableInterrupt = false
59 };
60 CTIMER_SetupMatch(CTIMERO_PERIPHERAL, CTIMERO_MATCH_3_CHANNEL, &CTIMERO_Match_3_config);
61 CTIMER_Reset(CTIMERO_PERIPHERAL);
62 CTIMER_StartTimer(CTIMERO_PERIPHERAL);
63 }
64
65
66 void set_state(int state_number){
67     switch (state_number) {
68         case 0:
69             GPIO_PinWrite(GPIO0, LED_RED_PIN, 1);
70             GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
71             GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);
72             break;
73         case 1:
74             GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
75             GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
76             GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);
77             break;
78         case 2:
79             GPIO_PinWrite(GPIO0, LED_RED_PIN, 1);
80             GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 1);
81             GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 0);
82             break;
83         case 3:
84             GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
85             GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 1);
86             GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 1);
87             break;
88         default:
89             GPIO_PinWrite(GPIO0, LED_RED_PIN, 0);
90             GPIO_PinWrite(GPIO0, LED_GREEN_PIN, 0);
91             GPIO_PinWrite(GPIO1, LED_BLUE_PIN, 0);
92             break;
93     }
94 }
95
96 /* GPIO00_IRQn interrupt handler */
97 void GPIO0_INT_0_IRQHANDLER(void) {
98     /* Get pin flags 0 */
99     uint32_t pin_flags0 = GPIO_GpioGetInterruptChannelFlags(GPIO0, 0U);
100
101     /* Place your interrupt code here */
102     state_counter = (state_counter + 1) % (sizeof(sample_vel) / sizeof(sample_vel[0]));
103     setup_new_match(sample_vel[state_counter]);
104     set_state(state_counter); /* Clear pin flags 0 */
105     GPIO_GpioClearInterruptChannelFlags(GPIO0, pin_flags0, 0U);
106
107     /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
108     Store immediate overlapping exception return operation might vector to incorrect
109     ↳ interrupt. */
110     #if defined __CORTEX_M && (__CORTEX_M == 4U)
111         __DSB();
112     #endif
113 }
114
115 /* GPIO01_IRQn interrupt handler */
116 void GPIO0_INT_1_IRQHANDLER(void) {
117     /* Get pin flags 1 */

```

```

117  uint32_t pin_flags1 = GPIO_GpioGetInterruptChannelFlags(GPIO0, 1U);
118
119  /* Place your interrupt code here */
120  flag_adc = ~flag_adc & 1;
121  /* Clear pin flags 1 */
122  GPIO_GpioClearInterruptChannelFlags(GPIO0, pin_flags1, 1U);
123
124  /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
125  Store immediate overlapping exception return operation might vector to incorrect
126  ↳ interrupt. */
127  #if defined __CORTEX_M && (__CORTEX_M == 4U)
128      __DSB();
129  #endif
130
131  void ADC0_IRQHANDLER(void) {
132      uint32_t trigger_status_flag;
133      uint32_t status_flag;
134      /* Trigger interrupt flags */
135      trigger_status_flag = LPADC_GetTriggerStatusFlags(ADC0_PERIPHERAL);
136      /* Interrupt flags */
137      status_flag = LPADC_GetStatusFlags(ADC0_PERIPHERAL);
138      /* Clears trigger interrupt flags */
139      LPADC_ClearTriggerStatusFlags(ADC0_PERIPHERAL, trigger_status_flag);
140      /* Clears interrupt flags */
141      LPADC_ClearStatusFlags(ADC0_PERIPHERAL, status_flag);
142
143      /* Place your code here */
144      static uint16_t sample_count = 0;
145      static uint16_t dac_count = 0;
146      lpadc_conv_result_t result;
147
148      if (LPADC_GetConvResult(ADC0_PERIPHERAL, &result, 0)) {
149          uint16_t v = (result.convValue >> 4) & 0xFFFF; /* solo si single-ended 12 bit */
150          uint16_t out;
151          if (flag_adc) { samples[sample_count] = (uint16_t)result.convValue; sample_count = (
152              ↳ sample_count + 1) % SAMPLES_NUM; out = v; }
153          else { out = (uint16_t)samples[dac_count]; dac_count = (dac_count + 1) %
154              ↳ SAMPLES_NUM; }
155          DAC_SetData(DAC0, out);
156      }
157
158      /* Add for ARM errata 838869, affects Cortex-M4, Cortex-M4F
159      Store immediate overlapping exception return operation might vector to incorrect
160      ↳ interrupt. */
161      #if defined __CORTEX_M && (__CORTEX_M == 4U)
162          __DSB();
163      #endif
164  }
165
166  /*
167  * @brief Application entry point.
168  */
169  int main(void) {
170
171      /* Init board hardware. */
172      BOARD_InitBootPins();
173      BOARD_InitBootClocks();
174      BOARD_InitBootPeripherals();
175      BOARD_InitLEDsPins();
176      BOARD_InitBUTTONsPins();
177
178      #ifndef BOARD_INIT_DEBUG_CONSOLE_PERIPHERAL
179          /* Init FSL debug console. */

```

```
176     BOARD_InitDebugConsole();
177 #endif
178
179     set_state(0);
180     DisableIRQ(ADCO_IRQn);
181     CTIMER_StopTimer(CTIMER0);
182     LPADC_DoOffsetCalibration(ADCO);
183     LPADC_DoAutoCalibration(ADCO);
184     LPADC_DoResetFIFO0(ADCO);
185     EnableIRQ(ADCO_IRQn);
186     CTIMER_StartTimer(CTIMER0);
187
188 //     /* Enter an infinite loop, just incrementing a counter. */
189     while(1) {
190     }
191     return 0 ;
192 }
```